

**Правительство Российской Федерации
Нижегородский филиал
Федерального государственного автономного образовательного
учреждения высшего образования
"Национальный исследовательский университет"
"Высшая Школа Экономики"**

**Факультет информатики, математики и компьютерных наук
Программа подготовки бакалавров по направлению
09.03.04 Программная инженерия**

Пичугин Артём Алексеевич

КУРСОВАЯ РАБОТА

**Разработка бэкенда кроссплатформенного приложения для владельцев
гражданского оружия DryFire**

**Научный руководитель:
Зотов Е.М.**

Нижний Новгород, 2026 г.

Аннотация

В теоритической части работы приводится описание предметной области, анализ существующих на рынке решений и актуальности разработки. В практической части работы представлен реальный процесс разработки серверной части приложения и продемонстрированы результаты в виде демонстрации работы API.

Ключевые слова

серверное приложение, REST API, Rust, Axum, PostgreSQL, SQLx, асинхронное программирование, гексагональная архитектура, чистая архитектура, доменно-ориентированное проектирование, аутентификация, Argon2, шифрование данных, AES-GCM, журнал аудита, полнотекстовый поиск, гражданское оружие, баллистический калькулятор, учёт боеприпасов, нормативно-правовые акты

Оглавление

Аннотация.....	2
Ключевые слова.....	3
Введение.....	5
Глава 1. Анализ предметной области для разработки приложения.....	7
1.1 Анализ актуальности разработки.....	7
1.2 Анализ существующих технологий разработки прикладного программного обеспечения.....	8
Глава 2. Разработка серверной части приложения.....	17
2.1 Функциональные требования.....	17
2.2 Разработка архитектуры серверной части приложения.....	22
2.2.1 Архитектурный стиль и принципы организации кода.....	23
2.2.2 Организация модулей.....	23
2.2.3 Используемые библиотеки.....	27
2.2.4 Безопасность.....	29
2.2.5 Версионирование API и расширяемость.....	30
2.3 Схема базы данных.....	30
2.3.1 Управление пользователями.....	31
2.3.2 Журнал аудита.....	32
2.3.3 Учёт оружия.....	33
2.3.4 Учёт документов (лицензии и разрешения).....	34
2.3.5 Учёт боеприпасов.....	35
2.3.6 Баллистические профили и профили прицелов.....	36
2.3.7 Нормативно-правовые акты.....	37
Заключение.....	40
Краткий глоссарий основных терминов.....	43
Архитектура и проектирование.....	43
Технологии разработки.....	43
Базы данных.....	44
Безопасность.....	45
Сеть и API.....	46
Предметная область (огнестрельное оружие).....	47
Баллистика и оптика.....	48
Список использованных литературных источников.....	50

Введение

В последние годы цифровизация государственных сервисов является одним из самых активно развивающихся направлений в Российской Федерации. Сфера оборота гражданского оружия не является исключением. Но всё же владельцы гражданского оружия часто сталкиваются с целым рядом проблем:

- необходимые законодательные акты находятся в нескольких источниках, часто обновляются и добавляются новые
- все необходимые для оружия документы выдаются на определенный срок, разными ведомствами и организациями, зачастую в разные даты, что увеличивает вероятность забыть вовремя подать заявку на продление

По последним актуальным данным от Росгвардии, на 2020 год количество единиц оружия составляло 6.5 миллионов, которыми владели 4.3 миллиона человек. Введение новых ограничений и ужесточение контроля может привести к снижению активных владельцев оружия и тем самым привести к снижению уровня оружейной культуры в стране.

Любому стрелку, охотнику или спортсмену важно контролировать расход боеприпасов, следить за сроками технического обслуживания оружия, записывать результаты тренировок и использовать различные инструменты, например баллистический калькулятор.

Разрабатываемая система призвана облегчить пользователю доступ ко всей необходимой владельцу оружия информации, контролировать свой арсенал, следить за изменениями в законодательстве и не пропустить сроки продления всех необходимых разрешений и лицензий.

Целью данной курсовой работы является разработка серверной части кроссплатформенного приложения «Dryfire» для владельцев гражданского оружия.

Для достижения целей, поставленных в данной работе решаются следующие задачи:

1. Провести анализ актуальности разработки.
2. Провести анализ технологий разработки программного обеспечения.

3. Провести анализ нормативно-правовой базы в сфере оборота гражданского оружия и API для программного взаимодействия с источниками нормативно-правовых актов.
4. Определить функционал разрабатываемого приложения.
5. Разработать серверную часть приложения.

Глава 1. Анализ предметной области для разработки приложения

1.1 Анализ актуальности разработки

На данный момент в сфере гражданского вооружения существует не так много приложений и сервисов. США является лидером по количеству огнестрельного оружия в частном владении, поэтому значительная доля приложений ориентирована на граждан этой страны. Так как законы об оружии там более свободные, то владельцам оружия не приходится сталкиваться с теми же проблемами, что испытывают граждане РФ. Так же есть приложения, созданные конкретно для баллистических расчетов, но из-за различий в системе мер создает неудобства для пользователей, привыкших к метрической системе. Так же их функционал ограничен: отсутствует учет документов и своего арсенала. Еще одной актуальной проблемой является недоступность западных сервисов на территории нашей страны без средств обхода блокировок, а так же невозможность оплатить какие-либо дополнительные функции с помощью российских платежных систем.

Любому владельцу гражданского оружия важные следующие факторы:

1. Растущее количество законов в сфере оборота гражданского оружия: несмотря на активную цифровизацию государственных ведомств, соблюдение всех необходимых сроков, оформление нужных документов, прохождение процедур требует глубокого погружения в законодательную базу, где легко может запутаться даже опытный владелец ружья. Наличие платформы, которое может автоматизировать часть этих процессов и упростить контроль за документами снизит стресс, возникающий из-за бюрократических проволочек.
2. Учет боеприпасов: Контроль количества израсходованных боеприпасов поможет запланировать покупку новых после проведенной стрелковой сессии или охоты. Так же у пользователя всегда под рукой отчет, который можно предоставить сотрудникам компетентных органов в случае какой-либо проверки условий хранения и использования оружия.
3. Периодический контроль состояния оружия.

4. Электронный инструментарий: умение пользоваться оружием не ограничивается только лишь умением нажать на спусковой крючок. Стрелок должен понимать баллистические характеристики своего оружия, боеприпасов, влияние погодных условий и прочих аспектов на выстрел.

Перечисленные выше аспекты указывают на то, что на сегодняшний момент разработка приложения имеет значительную актуальность для владельцев гражданского оружия.

1.2 Анализ существующих технологий разработки прикладного программного обеспечения

Проведение анализа существующих технологий разработки прикладного программного обеспечения имеет ряд причин и важных преимуществ:

1. Выбор оптимальных решений: Анализ позволяет изучить различные технологические варианты и выбрать наиболее подходящие для конкретного проекта. Это позволяет оптимизировать разработку и выбрать технологии, которые наилучшим образом соответствуют требованиям предприятия и проекта.

2. Определение совместимости: Результаты анализа современных информационных технологий могут быть использованы при выборе технических средств разработки информационной системы с учётом их наилучшей совместимости с уже существующей инфраструктурой предприятия. Это помогает избежать проблем с интеграцией различных модулей системы и обеспечить их гармоничное взаимодействие.

3. Оценка производительности и масштабируемости: Анализ позволяет изучить производительность и масштабируемость технологий в контексте разрабатываемого приложения. Это важно для обеспечения эффективной работы системы при увеличении объёма данных и пользовательской нагрузки.

4. Оптимизация затрат: Проведение анализа технологий помогает оценить стоимость разработки и поддержки приложения. Выбор оптимальных технологий может сократить расходы на разработку, интеграцию и обслуживание системы, а также обеспечить лучшее соотношение стоимости и качества.

5. Инновации и развитие: Анализ существующих технологий позволяет быть в курсе последних инноваций и тенденций в области разработки прикладного программного обеспечения. Это помогает выбрать современные и перспективные решения, которые позволят предприятию оставаться конкурентоспособным на рынке и готовым к будущим вызовам.

В целом, проведение анализа существующих технологий разработки прикладного программного обеспечения является неотъемлемой частью процесса разработки, которая помогает выбрать оптимальные решения, повысить эффективность и качество проекта, а также снизить риски и затраты.

Разработка серверной части приложения осуществляется непосредственно с помощью специального ПО, а именно средств разработки, редакторов кода и других вспомогательных инструментов. Серверная часть приложения состоит из слоя бизнес-логики (инструментальное средство для написания алгоритмов реакции приложения на то или иное действие) и слоя доступа к данным (хранение, выборка, изменение и удаление данных). Для слоя бизнес-логики необходимо выбрать платформу для разработки сервиса, а для слоя доступа к данным выбрать наиболее подходящую по критериям, отвечающим разработке данного ПО СУБД.

Для анализа лучшей СУБД были выявлены следующие группы критериев:

1. Структура данных
2. Функциональные возможности
3. Особенности разработки приложений

4. Производительность

5. Требования к рабочей среде

Приведены описания для каждой вышеперечисленной группы критериев:

1. Структура данных. К данной группе требований можно отнести следующие: используемая модель данных (вопрос об использовании той или иной модели должен решаться на начальном этапе проектирования информационной системы), предусмотренные типы данных.

2. Функциональные возможности. В данной группе требований выделяются следующие критерии: мобильность (независимость системы от среды, в которой она работает), масштабируемость (сможет ли данная система соответствовать росту информационной системы), триггеры и хранимые процедуры, сетевые возможности.

3. Особенности разработки приложений. Многие производители СУБД выпускают также средства разработки приложений для своих систем. Как правило, эти средства позволяют наилучшим образом реализовать все возможности сервера, поэтому при анализе СУБД стоит рассмотреть также и возможности средств разработки приложений. К данной группе требований можно отнести следующие: средства проектирования, многоязыковая поддержка, возможности разработки веб-приложений.

4. Производительность. Производительность системы является одним из самых важных показателей, который будет использоваться в качестве основного критерия для выбора СУБД. Существует несколько факторов, которые можно отнести к производительности системы: рейтинг TPC (Transactions per Cent), возможности параллельной архитектуры, возможности оптимизирования запросов.

5. Требования к рабочей среде. К данной группе требований можно отнести следующие: поддерживаемые аппаратные платформы, минимальные требования к оборудованию и операционной системе.

Таблица 1

Сравнительная характеристика СУБД

Структура данных	Функциональные возможности	Особенности разработки приложений	Производительность	Требования к рабочей среде
DB2	–	+	–	+
Oracle	+	–	–	–
MySQL	–	+	+	+
MS SQL	–	–	+	+
PostgreSQL	+	+	+	+

Как видно из таблицы, PostgreSQL продемонстрировал наилучшие результаты по совокупности критериев, получив положительные оценки по четырём из пяти групп. Это делает его оптимальным выбором СУБД для разрабатываемого приложения.

В качестве веб-фреймворка для разработки серверной части приложения могут выступать такие технологии как:

Actix Web – высокопроизводительный веб-фреймворк на языке Rust, основанный на акторной модели. Обеспечивает асинхронную обработку запросов и высокую скорость работы, однако имеет более сложный API и крутую кривую обучения по сравнению с конкурентами;

Rocket – веб-фреймворк на языке Rust, ориентированный на удобство разработки и безопасность типов. Предоставляет декларативный макро-синтаксис для определения маршрутов, однако в ранних версиях требовал использования

ночной сборки компилятора и имеет меньшее сообщество по сравнению с другими решениями;

Axum – модульный и эргономичный веб-фреймворк на языке Rust, разработанный командой Tokio. Построен на основе экосистемы Tower, что обеспечивает высокую совместимость с другими библиотеками. Axum использует систему типов Rust для гарантирования корректности обработки запросов на этапе компиляции и предоставляет удобные средства для маршрутизации, извлечения данных из запросов и обработки ошибок;

Warp – легковесный веб-фреймворк на языке Rust, основанный на концепции фильтров. Обеспечивает компонуемую архитектуру обработки запросов, но функциональный стиль программирования может усложнять отладку и понимание кода для начинающих разработчиков.

По результатам анализа существующих технологий разработки для приложения «Dryfire», соответствующих требованиям для разработки приложения, был выбран следующий стек технологий: Rust в качестве основного языка программирования, Axum для разработки веб-приложения и PostgreSQL для создания базы данных. Выбранные технологии обладают несколькими преимуществами:

1. Rust: является системным языком программирования, обеспечивающим безопасность работы с памятью без использования сборщика мусора. Система владения (ownership) и заимствования (borrowing) в Rust позволяет предотвращать ошибки, связанные с обращением к освобождённой памяти, гонками данных и утечками ресурсов, на этапе компиляции. Rust обеспечивает производительность, сопоставимую с C и C++, при значительно более высоком уровне безопасности кода. Кроме того, Rust имеет развитую экосистему пакетов (crates), систему сборки Cargo и активное сообщество разработчиков.

2. Axum: является современным асинхронным веб-фреймворком, разработанным командой Tokio – ведущей асинхронной среды выполнения для Rust. Axum построен на основе библиотеки Tower, что позволяет использовать богатую экосистему промежуточного ПО (middleware) для логирования, аутентификации, ограничения частоты запросов и других задач. Фреймворк предоставляет типобезопасную маршрутизацию, удобные экстракторы для извлечения данных из запросов (параметры пути, тело запроса, заголовки) и встроенную поддержку WebSocket-соединений. Благодаря интеграции с Tokio, Axum обеспечивает высокую пропускную способность при обработке параллельных запросов.

3. PostgreSQL: выделяется в качестве мощной системы управления базами данных, особенно в обработке больших объёмов данных с исключительной скоростью. Оптимизированная архитектура позволяет обрабатывать тысячи запросов в секунду, что делает PostgreSQL сильным претендентом для требовательных к СУБД приложений. PostgreSQL предлагает комплексный набор функций, включая поддержку JSON-типов данных, полнотекстового поиска, транзакций с уровнями изоляции и расширяемую систему типов, что обеспечивает широкий спектр возможностей для сложных проектов.

Выбор данных технологий обусловлен сочетанием их преимуществ, которые являются критическими для успешной разработки приложения «Dryfire». Они обеспечивают надёжность, высокую производительность, безопасность и масштабируемость. Комбинация Rust, Axum и PostgreSQL позволит эффективно реализовать функциональность приложения «Dryfire», обеспечивая высокую скорость обработки запросов, безопасную работу с памятью, а также надёжное взаимодействие с базой данных.

Для реализации данного приложения наряду с выбранным стеком технологий была выбрана среда разработки Visual Studio Code, поскольку она имеет ряд существенных преимуществ:

1. Легковесная среда разработки: Visual Studio Code представляет собой легковесный, но мощный редактор кода, который обеспечивает быстрый запуск и отзывчивый интерфейс. В отличие от полноценных IDE, VS Code потребляет значительно меньше системных ресурсов, при этом предоставляя необходимую функциональность для разработки: подсветку синтаксиса, автодополнение, интегрированный терминал, систему управления версиями и встроенный отладчик.

2. Поддержка языка Rust: Visual Studio Code обеспечивает превосходную поддержку языка Rust через расширение rust-analyzer, которое предоставляет интеллектуальное автодополнение, проверку типов в реальном времени, навигацию по коду, рефакторинг и встроенные подсказки. Расширение использует возможности системы типов Rust для предоставления точных и контекстно-зависимых рекомендаций.

3. Расширяемая архитектура: VS Code обладает богатой экосистемой расширений, доступных через встроенный магазин. Для разработки на Rust доступны расширения для отладки (CodeLLDB), форматирования кода, управления зависимостями Cargo, работы с базами данных PostgreSQL и другие инструменты, что позволяет настроить среду разработки под конкретные потребности проекта.

4. Кроссплатформенность: Visual Studio Code доступен на всех основных операционных системах (Windows, macOS, Linux), что обеспечивает единообразный опыт разработки независимо от платформы. Это особенно важно

при работе с Rust, который также является кроссплатформенным языком программирования.

5. Интеграция с системами контроля версий: VS Code предоставляет встроенную поддержку Git, включая визуальное отображение изменений, управление ветками, просмотр истории коммитов и разрешение конфликтов слияния. Это упрощает совместную работу над проектом и управление версиями кода.

Таким образом, ведение разработки с использованием выбранных технологий в Visual Studio Code обеспечивает удобную и эффективную среду для разработки серверной части приложения, поддерживает выбранный стек технологий, предоставляет доступ к обширной экосистеме расширений и обеспечивает кроссплатформенную разработку.

Для взаимодействия приложения на Rust с базой данных PostgreSQL, а также для решения сопутствующих задач были выбраны следующие вспомогательные библиотеки:

SQLx – асинхронная библиотека для работы с базами данных на языке Rust, поддерживающая проверку SQL-запросов на этапе компиляции. SQLx обеспечивает безопасное взаимодействие с PostgreSQL без необходимости использования ORM, позволяя писать чистые SQL-запросы с гарантией их корректности;

Tokio – асинхронная среда выполнения для Rust, обеспечивающая многопоточную обработку ввода-вывода. Tokio является основой для работы Axum и SQLx, предоставляя планировщик задач, примитивы синхронизации и асинхронные сетевые операции;

Serde – фреймворк для сериализации и десериализации данных в Rust. Serde обеспечивает автоматическое преобразование структур данных в формат JSON и обратно, что необходимо для работы с API;

jsonwebtoken – библиотека для создания и проверки JWT-токенов (JSON Web Token), используемых для аутентификации и авторизации пользователей в приложении.

Глава 2. Разработка серверной части приложения

2.1 Функциональные требования

Исходя из проблем владельцев гражданского оружия, были составлены следующие требования к серверной части приложения:

- 1 Управление пользователями, аутентификация и безопасность
 - 1.1 Сервер должен принимать учетные данные от клиентского приложения, валидировать их и при успешной авторизации генерировать и возвращать защищенный токен сессии.
 - 1.2 Учетные данные должны включать в себя: имя, регион, язык
 - 1.3 При регистрации нового аккаунта должна генерировать уникальную ссылку с ограниченным сроком действия и отправлять её пользователю для верификации через интеграцию с почтовым сервисом или верифицировать сгенерированный ключ в OAuth 2.0.
 - 1.4 Система должна обеспечивать шифрование пользовательских чувствительных данных с помощью современных алгоритмов шифрования.
 - 1.5 Система должна хранить данные о последнем визите пользователя.
- 2 Обработка данных
 - 2.1 Бэкенд должен осуществлять валидацию всех входных данных, чтобы предотвратить сохранение некорректной информации или попытки внедрения вредоносного кода, возвращать ошибку в случае, если клиентское приложение не отправило все обязательные данные.
 - 2.2 Сервер должен принимать загружаемые мультимедийные файлы (сканы разрешений, фотографии), безопасно размещать их в облачном объектном хранилище и связывать ссылки на эти документы со строками в реляционной базе данных конкретного пользователя.
- 3 Справочник нормативно-правовых актов
 - 3.1 Система должна хранить тексты законов, относящихся к обороту оружия, в зависимости от региона пользователя

- 3.2 Система должна в фоновом режиме получать обновление законов и добавление новых.
- 3.3 Система должны поддерживать версионирование документов.
- 3.4 Система должна фильтровать документы по их назначению (транспортировка, ношение, хранение, охота, самооборона и т. п.)
- 3.5 Система должна поддерживать полнотекстовый поиск по документам.
- 3.6 Система должна поддерживать навигацию по категориям\тематикам нормативно-правовых актов.
- 3.7 Система должна предоставить API для получения данных о последних изменениях в законах с момента последнего визита пользователя.

4 Учет инвентаря

- 4.1 Система должна давать пользователю возможность создавать, изменять, удалять записи об оружии.
- 4.2 Любая запись должна включать: производителя, модель, серийный номер (зашифрованные в месте хранения), тип, дату приобретения и опционально фото.
- 4.3 Система должна поддерживать возможность связывать записи об оружии с записями о лицензиях.
- 4.4 Система должна поддерживать каталог с типичными моделями огнестрельного оружия с их заводскими характеристиками (длина стволов, вес, используемый калибр и т.д.)

5 Учет документов

- 5.1 Система должна давать пользователю возможность создавать, редактировать, удалять записи о лицензиях.
- 5.2 Записи о лицензиях должны содержать следующую информацию: место выдачи (организация), дата выдачи, дата окончания срока действия. Для некоторых документов дата окончания срока действия рассчитывается автоматически (на основании нормативно-правовых актов).

5.3 Система должна помечать лицензии с истекшим сроком действия.

5.4 Сервер должен хранить данные об истечении срока годности лицензий и разрешений и посылать push-уведомления, уведомления на электронную почту за 90, 60, 45, 30, 14 дней до срока истечения.

5.5 Система должна предоставлять API для получения всех дедлайнов в заданном пользователем промежутке

5.6 Система должна хранить и инструкции по оформлению и продлению документов.

6 Учет аммуниции

6.1 Система должна позволять пользователям отслеживать боеприпасы с фильтрами по производителю, калибру, типу пули, типу снаряда, навески пороха.

6.2 Система должна поддерживать ручные операции с количеством, временем и опциональным полем для заметок.

6.3 Система должна вести учет всех транзакций с количеством патронов.

6.4 Система должна агрегировать суммарное количество патронов по различным атрибутам.

6.5 Система должна вести статистику по расходу: по калибрам, по оружию, за определенный временной период.

6.6 Бэкенд должен предоставлять API для с историей использования и статистикой.

7 Баллистические расчеты

7.1 Система должна принимать следующие баллистические параметры: калибр, вес пули, начальная скорость, баллистический коэффициент, высота прицела над осью ствола, дистанция пристрелки.

7.2 Система должна принимать параметры окружающей среды: температура, высота/давление, влажность, скорость ветра, направление ветра.

- 7.3 Система должна позволять пользователям сохранять и повторно использовать именованные баллистические профили (привязанные к конкретной комбинации оружие + боеприпас)
- 7.4 Система должна возвращать данные траектории в виде структурированной таблицы (дистанция, снижение, боковой снос, скорость, энергия, время полёта).
- 7.5 Система должна предоставлять значения поправок вверх/вниз (МОА или MIL) на каждом шаге дистанции для корректировки прицела.
- 7.6 Система должна поддерживать экспорт таблиц траектории в форматах JSON и CSV.
- 8 Настройка прицела
 - 8.1 Система должна принимать на вход: наблюдаемое отклонение средней точки попадания (по горизонтали и вертикали) на заданной дистанции, единицу настройки прицела (МОА или MIL), цену клика оптики.
 - 8.2 Система должна рассчитывать необходимое количество кликов (по вертикали и горизонтали) для перемещения средней точки попадания в желаемую точку пристрелки.
 - 8.3 Система должна учитывать разницу между истинной МОА (1,047" на 100 ярдов) и стрелковой МОА (1" на 100 ярдов), когда это имеет значение.
 - 8.4 Система должна рассчитывать поправку, необходимую для перепристрелки оружия с одной дистанции на другую, используя привязанный баллистический профиль.
 - 8.5 Система должна позволять пользователям хранить профили оптики: название, тип настройки (МОА/MIL), цена клика, максимальный диапазон вертикальных/горизонтальных поправок, высота установки прицела.
 - 8.6 Система должна привязывать профили оптики к конкретным единицам оружия.
- 9 Безопасность данных и конфиденциальность

9.1 Всё взаимодействие через API должно осуществляться по протоколу TLS 1.2+.

9.2 Чувствительные поля (серийные номера, сканы лицензий, персональные документы) должны быть зашифрованы при хранении.

9.3 Система должна вести журнал аудита всех операций доступа к данным и их изменения.

9.4 Система должна обеспечивать ограничение частоты запросов (rate limiting) и защиту от перебора на эндпоинтах аутентификации.

10 Административный API и управление контентом

10.1 Система должна предоставлять административные эндпоинты для управления библиотекой правового контента (CRUD-операции над правовыми документами).

10.2 Система должна предоставлять административные эндпоинты для управления справочным каталогом оружия.

10.3 Система должна предоставлять административные эндпоинты для просмотра агрегированной обезличенной аналитики использования (активные пользователи, использование функций).

10.4 Система должна предоставлять административные эндпоинты для управления конфигурацией, специфичной для юрисдикции (правила продления, лимиты боеприпасов).

11 Интеграция и проектирование API

11.1 Backend должен предоставлять RESTful API.

11.2 Система должна поддерживать версионирование API через путь URL.

11.3 Все эндпоинты, возвращающие списки, должны поддерживать пагинацию, сортировку и фильтрацию.

11.4 Система должна возвращать стандартизированные ответы об ошибках с кодами ошибок и человекочитаемыми сообщениями.

2.2 Разработка архитектуры серверной части приложения.

Серверная часть приложения является ядром всей системы, так как в ней содержится основная бизнес-логика приложения, связываются все компоненты, такие как клиентские приложения, системы хранения данных, авторизация, push-уведомлений и т. п.

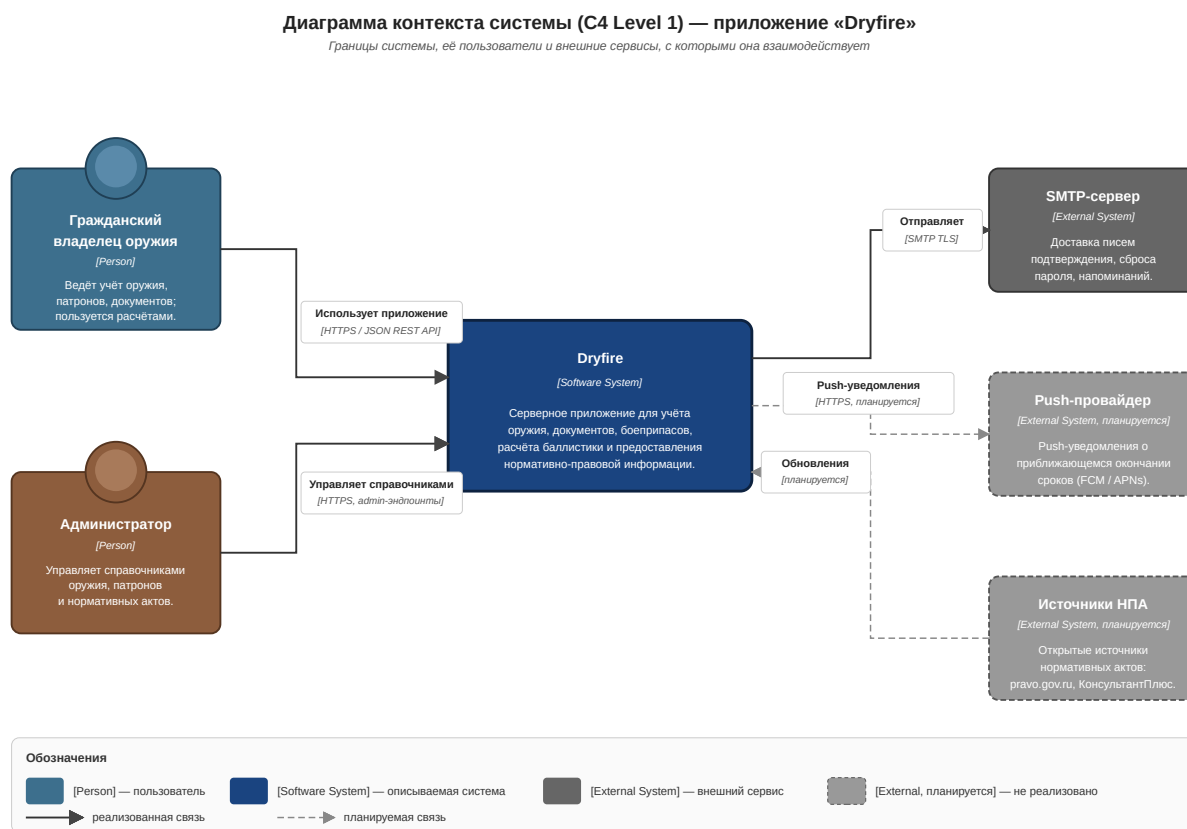


Рисунок 1. Диаграмма C4 приложения Dryfire

2.2.1 Архитектурный стиль и принципы организации кода

Серверная часть построена по принципам **гексагональной (портово-адаптерной) архитектуры**, также известной как Clean Architecture. Этот подход разделяет приложение на концентрические слои, в которых направление зависимостей строго направлено внутрь — от внешних слоёв (HTTP, СУБД) к внутреннему ядру (бизнес-логика). Такая организация позволяет:

1. **Изолировать бизнес-логику** от деталей реализации инфраструктуры. Доменный слой не знает ни о существовании HTTP, ни о конкретной СУБД, ни о выбранной библиотеке шифрования.

2. **Заменять компоненты инфраструктуры без изменения бизнес-логики.** Например, переход с PostgreSQL на другую реляционную СУБД, замена SMTP-провайдера или алгоритма хеширования паролей затрагивает только один слой кода.

3. **Облегчить модульное тестирование.** Поскольку зависимости передаются через трейты (интерфейсы), в тестах легко подставить мок-реализации репозитория и внешних сервисов.

4. **Структурировать рост кодовой базы.** Каждая предметная область (пользователи, оружие, документы, боеприпасы, баллистика, прицел, нормативно-правовые акты) реализована по одинаковому шаблону: сущность → репозиторий → варианты использования → контроллер.

Проект организован как Cargo-крейт с библиотечной и бинарной частями. Логика приложения вынесена в библиотечный крейт (`lib.rs`), а бинарный (`main.rs`) содержит лишь точку входа: инициализацию переменных окружения, настройку подсистемы логирования и вызов функции запуска приложения. Такое разделение позволяет писать интеграционные тесты, обращающиеся к приложению как к библиотеке, без необходимости запускать отдельный процесс.

2.2.2 Организация модулей

Кодовая база разделена на четыре основных модуля верхнего уровня, расположенных в каталоге `src/`:

1. **Модуль `domain`** — доменный слой, содержащий чистые типы предметной области и определения интерфейсов (трейтов). Не содержит ни

одной зависимости от веб-фреймворка, СУБД или асинхронного ввода-вывода. Внутри модуля выделены подмодули:

- `entities/` — сущности и агрегаты: `User`, `Gun`, `License`, `AmmoType`, `AmmoTransaction`, `BallisticProfile`, `ScopeProfile`, `Law`, `LicenseType`, `CatalogEntry` и др. Каждая сущность инкапсулирует свои инварианты и предоставляет конструкторы-валидаторы (так называемые "smart constructors"). Например, конструктор `User::register` отклоняет регистрацию пользователей младше совершеннолетнего возраста, а `Password::new` — слабые пароли, не удовлетворяющие политике сложности.

- `repositories/` — трейты, описывающие контракты хранилищ данных: `UserRepository`, `GunRepository`, `LicenseRepository`, `AmmoRepository`, `LawRepository` и т. д. Каждый трейт перечисляет операции, которые приложение может выполнять с соответствующим агрегатом, не описывая реализацию.

- `services/` — трейты внешних сервисов и криптографических примитивов: `Hasher` (медленное хеширование паролей), `Signer` (быстрая подпись токенов), `FieldCipher` (шифрование чувствительных полей), `TokenHandler` (генерация и валидация токенов сессий), `Mailer` (отправка писем), `AuditLogger` (запись в журнал аудита).

- `errors.rs` — типизированные ошибки доменного слоя: `DomainError` (бизнес-ошибки) и `ValidationError` (ошибки валидации входных данных). Использование типизированных ошибок вместо обобщённого `anyhow::Error` позволяет вышестоящим слоям точно различать причины сбоя и сопоставлять их с HTTP-статусами.

2. Модуль **application** — слой вариантов использования (use cases).

Содержит классы-оркестраторы, реализующие конкретные бизнес-сценарии: регистрация пользователя, вход в систему, добавление оружия, проведение транзакции с боеприпасами, расчёт баллистической траектории и т. п. Каждый вариант использования принимает зависимости (репозитории и

сервисы) через структуру разделяемого состояния `AppState` и определяет одну точку входа `execute()`. Подмодули:

- `app_state.rs` — определение структуры `AppState`, содержащей разделяемые компоненты приложения: пул подключений к СУБД, конфигурацию, реализации всех трейтов в виде `Arc<dyn Trait>`. Структура реализует `Clone` с константной сложностью за счёт использования атомарных счётчиков ссылок.
- `user_use_cases.rs`, `armory_use_cases.rs`, `license_use_cases.rs`, `ammo_use_cases.rs`, `ballistics_use_cases.rs`, `scope_use_cases.rs`, `law_use_cases.rs` — наборы вариантов использования по предметным областям.

3. **Модуль `infra`** — инфраструктурный слой, содержащий конкретные реализации трейтов из доменного слоя. Подмодули:

- `config.rs` — чтение конфигурации из переменных окружения, валидация её корректности при запуске.
- `db/` — пул подключений к PostgreSQL и реализации репозитория (PgUserRepository, PgGunRepository, PgLicenseRepository, PgAmmoRepository, PgLawRepository и др.). Каждый репозиторий выполняет SQL-запросы через библиотеку SQLx, отображая строки результата на сущности доменного слоя.
- `hash/` — реализации криптографических примитивов: ArgonHasher (Argon2id для хеширования паролей), HmacSigner (HMAC-SHA256 для подписи токенов), AesGcmCipher (AES-256-GCM для шифрования чувствительных полей при хранении).
- `mail/` — реализация интерфейса Mailer. На текущем этапе реализована LoggingMailer, выводящая сообщения в журнал; в продакшен-окружении заменяется на SMTP-реализацию на базе библиотеки lettre.
- `scheduler/` — фоновые задачи (background tasks). На текущий момент содержит планировщик license_reminders, опрашивающий базу данных с настраиваемым интервалом и

отправляющий уведомления о приближающемся окончании срока действия документов за 90, 60, 45, 30 и 14 дней до истечения.

- `server.rs` — трейт `Server`, определяющий контракт HTTP-сервера, и точка входа `init_app()`, выполняющая инициализацию приложения: загрузка конфигурации, создание пула подключений, запуск миграций, инстанцирование всех зависимостей и передача управления HTTP-серверу.

4. **Модуль `controller`** — слой входящих адаптеров. Содержит подмодуль `http/` с реализацией HTTP-API на базе фреймворка `Axum`:

- `http.rs` — корневой роутер, объединяющий поддеревья маршрутов всех предметных областей и применяющий слои `middleware` (трассировка, ограничение размера тела запроса, тайм-аут, CORS, генерация идентификатора запроса).

- `errors.rs` — конвертация типизированных доменных ошибок в стандартизированные HTTP-ответы с кодами ошибок и человекочитаемыми сообщениями.

- `middleware/` — `middleware`-функции: проверка авторизации (`require_auth`), проверка административных привилегий (`require_admin`), генерация и проброс идентификатора запроса (`set_request_id`).

- `user/`, `armory/`, `license/`, `ammo/`, `ballistics/`, `scope/`, `law/` — поддеревья маршрутов для каждой предметной области. Внутри каждого подмодуля содержатся `payload.rs` (DTO-структуры для запросов и ответов) и `router.rs` (определения маршрутов и обработчики).

5. **Модуль `utils`** — вспомогательные утилиты: `b64` (кодирование `base64url`), `time` (работа с метками времени), `paging` (общие примитивы пагинации, сортировки и фильтрации), `tokengenerator` (генерация и валидация подписанных токенов сессий).

2.2.3 Используемые библиотеки

Для реализации серверной части используются следующие основные библиотеки экосистемы Rust:

1. **Axum** — современный, асинхронный веб-фреймворк, построенный на базе библиотеки tower и асинхронной среды выполнения tokio. Основа для создания HTTP-API. Axum предоставляет типобезопасный механизм извлечения параметров запроса (extractors), композицию обработчиков, маршрутизацию по статическим и динамическим путям, а также интеграцию с экосистемой middleware tower-http.

2. **SQLx** — асинхронная библиотека для управления SQL-запросами. Не зависит от среды выполнения и работает напрямую с PostgreSQL через нативный протокол, без использования ORM. Поддерживает проверку корректности SQL-запросов во время компиляции (для статических запросов) и параметризованные запросы (для динамически собираемых), что исключает возможность SQL-инъекций.

3. **Tokio** — асинхронная среда выполнения, обеспечивающая многопоточность и неблокирующий ввод-вывод. Используется как фундамент для всего сетевого взаимодействия и работы с СУБД. Для тяжёлых синхронных операций (например, хеширование пароля по алгоритму Argon2) используется `tokio::task::spawn_blocking`, что исключает блокировку рабочих потоков обработки запросов.

4. **Serde** и **serde_json** — сериализация и десериализация JSON. Используется для всех HTTP-запросов и ответов, а также для хранения структурированных данных в JSONB-колонках PostgreSQL (например, в баллистических профилях).

5. **Argon2** — реализация алгоритма Argon2id, рекомендованного OWASP для хеширования паролей. Применяется при регистрации пользователя и проверке учётных данных при входе.

6. **HMAC** и **SHA-2** — реализация быстрого аутентифицирующего кода (HMAC-SHA256), используемого для подписи токенов сессий. Сознательно отделён от медленного хеширования паролей: применение Argon2 для проверки токена при каждом запросе привело бы к недопустимым задержкам.

7. **AES-GCM** — реализация симметричного аутентифицированного шифрования AES-256-GCM. Применяется для шифрования чувствительных полей при хранении в БД (серийные номера оружия, сканы лицензий).

8. **Secrecy** — обёртки для секретных значений (паролей, ключей), исключающие их случайное попадание в журналы или вывод отладочной информации.

9. **Subtle** — реализация операций сравнения с постоянным временем выполнения. Используется при проверке HMAC-подписей токенов для защиты от атак по времени.

10. **Chrono** — работа с датами и временем в форматах, совместимых с серверным и клиентским ПО (ISO 8601, RFC 3339).

11. **UUID** — генерация и сериализация универсально уникальных идентификаторов версии 4, используемых в качестве первичных ключей всех сущностей.

12. **Tower** и **tower-http** — компонуемые слои промежуточной обработки (middleware): трассировка запросов, ограничение размера тела, тайм-ауты, политика CORS.

13. **Tracing** и **tracing-subscriber** — структурированное логирование с поддержкой иерархии вызовов (spans) и автоматической корреляции записей журнала с идентификатором запроса.

14. **Thiserror** и **anyhow** — типизированные перечисления ошибок (для доменного слоя) и обобщённый тип ошибки (для инфраструктурных операций) соответственно.

15. **Validator** — декларативная валидация полей DTO-структур с использованием атрибутов.

16. **Regex** — регулярные выражения для валидации форматов (email, логин, пароль).

17. **Lettre** — отправка электронных писем по протоколу SMTP с поддержкой TLS.

18. **Rand** — генерация криптографически стойких случайных значений для одноразовых токенов верификации электронной почты и сброса пароля.

19. **Ipnetwork** — типизированная работа с IP-адресами при сохранении их в колонках типа INET PostgreSQL (журнал аудита, история сессий).

20. **Envy** и **dotenvy** — чтение конфигурации из переменных окружения и из файла `.env` в режиме разработки.

21.**Swagger / OpenAPI** — автогенерирование документации API по спецификации OpenAPI 3.0 (планируется к подключению в финальной фазе разработки через крейт `utoipa`).

2.2.4 Безопасность

Архитектура учитывает требования к безопасности, перечисленные в техническом задании:

1. **Хеширование паролей** алгоритмом Argon2id с автоматически генерируемой солью. Хеши хранятся в отдельной таблице `user_credentials`, изолированной от профиля пользователя.

2. **Шифрование чувствительных полей** при хранении: серийные номера оружия и сканы лицензий шифруются AES-256-GCM с уникальным поппсе на каждую операцию записи. Ключ шифрования загружается из переменной окружения и никогда не передаётся в БД.

3. **Подпись токенов сессий** через HMAC-SHA256 с использованием серверного ключа. Срок действия токена ограничен; помимо криптографической проверки, действительность сессии дополнительно проверяется по таблице `user_sessions` (что позволяет отзывать сессии).

4. **Защита от атак по времени** в обработчике входа: проверка пароля выполняется даже при отсутствии пользователя с указанным логином (по специальному фиксированному хешу-заглушке), чтобы время отклика не позволяло определить факт существования аккаунта.

5. **Защита от SQL-инъекций** обеспечивается параметризованными запросами SQLx; пользовательский ввод никогда не интерполируется в строку SQL.

6. **Изоляция данных по пользователю** на уровне репозиториев: каждый запрос к данным, принадлежащим пользователю, содержит условие `WHERE user_id = $1`, что исключает несанкционированный доступ к чужим записям.

7. **Журнал аудита** — отдельная таблица `audit_log` фиксирует все изменения пользовательских данных с указанием идентификатора пользователя, действия, затронутого ресурса, IP-адреса и идентификатора запроса.

8. Идентификатор запроса (`x-request-id`) — каждому входящему запросу присваивается UUID, который сопровождает все записи журнала и возвращается клиенту в HTTP-заголовке ответа, что облегчает расследование инцидентов.

9. Стандартизированный формат ошибок в формате JSON с кодом, человекочитаемым сообщением и идентификатором запроса. Внутренние ошибки сервера никогда не возвращают клиенту детали о причинах сбоя.

2.2.5 Версионирование API и расширяемость

Версионирование API осуществляется через путь URL (`/api/v1/...`), что позволяет в дальнейшем вводить новые версии API без нарушения совместимости с существующими клиентами. Каждая предметная область смонтирована как поддерево корневого роутера: `/api/v1/users`, `/api/v1/armory`, `/api/v1/licenses`, `/api/v1/ammo`, `/api/v1/ballistics`, `/api/v1/scope`, `/api/v1/laws`. Административные эндпоинты вынесены в подпрефикс `/admin/` соответствующих предметных областей и защищены отдельным middleware `require_admin`.

2.3 Схема базы данных

В качестве СУБД используется **PostgreSQL**. Выбор обусловлен следующими возможностями:

1. Развитая поддержка типов данных: UUID, JSONB, массивы (`TEXT[]`), геометрические типы, типы для IP-адресов (`INET`, `CIDR`), пользовательские перечисления (`ENUM`).

2. Полнотекстовый поиск через типы `tsvector` и `tsquery` с поддержкой стеммеров для русского и английского языков, используемый для поиска по нормативно-правовым актам.

3. GIN-индексы для эффективной фильтрации по массивам и полнотекстового поиска.

4. Триггеры на уровне БД, использованные для автоматического версионирования законов и для поддержания материализованного агрегата текущих остатков боеприпасов.

5. Транзакционная целостность при многотабличных операциях (например, при регистрации лицензии с привязкой к нескольким единицам оружия).

Миграции схемы базы данных хранятся в каталоге `migrations/` в виде SQL-файлов с порядковыми номерами и применяются автоматически при запуске приложения через библиотеку SQLx. Каждая миграция содержит DDL-инструкции для создания одной логически связанной группы таблиц, индексов, триггеров и ограничений целостности.

Схема базы данных приложения построена на принципах нормализации до третьей нормальной формы.

Полная схема включает в себя **18 таблиц, 3 пользовательских перечисляемых типа (ENUM)**, а также набор индексов и триггеров, обслуживающих специфические сценарии доступа к данным. Ниже приведено описание схемы, сгруппированное по предметным областям.

2.3.1 Управление пользователями

Подсистема управления пользователями реализована четырьмя таблицами.

Таблица `users` хранит публичный профиль пользователя: идентификатор, логин, имя, фамилию, электронную почту, дату рождения, регион проживания, язык интерфейса, статус учётной записи, признак административных привилегий, метку времени последнего визита, а также служебные метки времени создания и обновления записи. В качестве типа для статуса учётной записи используется пользовательский ENUM-тип `user_status` со значениями `pending`, `verified`, `blocked`. Уникальность логина и электронной почты обеспечивается двумя уникальными индексами по выражениям `LOWER(login)` и `LOWER(email)` — это позволяет нечувствительный к регистру поиск без необходимости приводить данные к нижнему регистру при каждом запросе. Для быстрого поиска административных пользователей создан частичный индекс `users_admin_idx`, охватывающий только записи с признаком `is_admin = TRUE`.

Таблица `user_credentials` содержит хеш пароля пользователя в формате PHC (стандартизированный формат, генерируемый алгоритмом Argon2id). Вынесение хеша в отдельную таблицу с тем же первичным ключом, что и `users`, позволяет считывать профиль пользователя без обращения к чувствительным аутентификационным данным.

Таблица `user_sessions` реализует серверное хранилище сессий. Каждая строка соответствует одной активной сессии: уникальный идентификатор сессии, ссылка на пользователя, метки времени создания, истечения и последней активности, опциональная метка времени отзыва (для отозванных сессий), строка User-Agent и IP-адрес клиента (тип `INET`). Серверное хранение сессий, в отличие от полностью stateless-подхода JWT, позволяет принудительно отзывать сессии (например, при смене пароля или подозрении на компрометацию).

Таблицы `email_verifications` и `password_resets` имеют идентичную структуру и хранят одноразовые токены для подтверждения адреса электронной почты и сброса пароля соответственно. Каждая запись содержит HMAC-хеш токена (никогда не сам токен в открытом виде), метку времени истечения и опциональную метку использования. Уникальное ограничение на колонку `token_hash` обеспечивает невозможность повторного использования. Разнесение этих двух функций по двум независимым таблицам позволяет в будущем менять политики (срок действия, частоту повторных запросов) для каждой функции независимо.

2.3.2 Журнал аудита

Таблица `audit_log` реализует требование технического задания о ведении журнала всех операций доступа к данным и их изменения. Это таблица типа append-only: записи в неё только добавляются, никогда не модифицируются и не удаляются. Каждая запись содержит автоинкрементный идентификатор (тип `BIGSERIAL`, рассчитанный на миллиарды записей), метку времени события, идентификатор пользователя (необязательный — для событий неаутентифицированных пользователей, например неудачных входов), машинно-читаемый код действия (`auth.login`, `gun.create`, `license.update` и т. п.), тип и идентификатор затронутого ресурса, идентификатор запроса для корреляции

с журналом сервера, IP-адрес клиента и поле произвольных метаданных в формате JSONB.

Для типичных запросов к журналу аудита созданы два составных индекса: `(user_id, occurred_at DESC)` — для запросов вида "что делал пользователь X между моментами Y и Z", и `(resource_type, resource_id)` — для запросов вида "что происходило с ресурсом R".

2.3.3 Учёт оружия

Подсистема учёта оружия включает две таблицы.

Таблица `gun_catalog` содержит справочник распространённых моделей огнестрельного оружия с заводскими характеристиками: производитель, модель, класс оружия (винтовка, ружьё, пистолет, револьвер, ПП, прочее), используемый калибр, длина ствола, масса, ёмкость магазина, произвольные примечания. Справочник управляется администраторами и доступен на чтение всем пользователям. Уникальное ограничение на пару `(manufacturer, model)` исключает дублирование. Индексы по калибру и классу обеспечивают эффективную фильтрацию при просмотре справочника.

Таблица `guns` хранит записи о собственном оружии пользователя. Помимо стандартных полей (производитель, модель, класс, калибр, дата приобретения, ссылка на фото, примечания, опциональная ссылка на запись справочника) таблица содержит две колонки для серийного номера:

- `serial_ciphertext` (тип `BYTEA`) — серийный номер в зашифрованном виде. Шифрование выполняется на стороне приложения алгоритмом AES-256-GCM с уникальным поппсе на каждую операцию записи. Ключ шифрования загружается из переменной окружения и никогда не передаётся в БД. Даже при полной компрометации базы данных серийные номера остаются недоступными без ключа.
- `serial_last4_plain` (тип `TEXT`) — последние четыре символа серийного номера в открытом виде, используемые для подсказок в интерфейсе ("какая из моих винтовок имеет серийный номер, заканчивающийся на ...1234?"), без необходимости расшифровывать каждую запись при отображении списка.

Этот подход реализует принцип "криптография для безопасности, открытый текст для производительности" в применении к одной и той же сущности.

Для типичных запросов созданы составные индексы (`user_id, class`) и (`user_id, caliber`), ускоряющие фильтрацию арсенала пользователя по этим атрибутам.

2.3.4 Учёт документов (лицензии и разрешения)

Подсистема учёта документов состоит из четырёх таблиц.

Таблица `license_types` — справочник типов документов, управляемый администраторами. Каждый тип содержит машинно-читаемый код, человекочитаемое название, регион действия, опциональный срок действия в днях (для автоматического расчёта даты окончания), а также текстовые инструкции по оформлению и продлению документа.

Таблица `licenses` содержит записи о документах пользователя: ссылку на тип документа, номер, выдавший орган, дату выдачи, дату окончания срока действия, примечания, ссылку на скан и, опционально, поле `scan_blob_ciphertext` для инлайн-хранения зашифрованного скана. Для типичных запросов созданы индексы по `user_id` и составной индекс (`user_id, expires_at`), обслуживающий запросы вида "мои документы, отсортированные по сроку истечения".

Таблица `license_guns` реализует связь "многие ко многим" между документами и единицами оружия: одна лицензия может покрывать несколько единиц оружия, и одна единица оружия может в разное время покрываться разными лицензиями. Первичный ключ таблицы — пара (`license_id, gun_id`). Каскадное удаление обеспечивает целостность при удалении документа или оружия.

Таблица `license_notifications` обслуживает планировщик уведомлений. Каждая запись фиксирует факт отправки одного уведомления для одной лицензии (поле `days_before` указывает, за сколько дней до истечения было отправлено уведомление: 90, 60, 45, 30 или 14). Уникальное ограничение на пару (`license_id, days_before`) обеспечивает идемпотентность: повторный запуск планировщика не приведёт к повторной отправке уже

отправленных уведомлений. При изменении даты окончания срока действия документа все ранее отправленные уведомления для этой лицензии удаляются, что позволяет планировщику корректно отработать новые сроки.

2.3.5 Учёт боеприпасов

Подсистема учёта боеприпасов реализована тремя таблицами и сопровождается двумя пользовательскими ENUM-типами.

ENUM-тип `bullet_type` определяет тип пули (FMJ — оболочечная, JHP — экспансивная с полостью, SP — мягкая, LRN — свинцовая круглоносая, WAD — пуля типа "вадкаттер", match — матчевая, slug — пулевой патрон гладкоствольного оружия, buckshot — картечь, birdshot — дробь, other — прочее). ENUM-тип `projectile_type` определяет тип патрона (centerfire — центрального воспламенения, rimfire — кольцевого воспламенения, shotshell — гладкоствольный, blank — холостой, other — прочее).

Таблица `ammo_types` — справочник типов боеприпасов, управляемый администраторами. Каждый тип содержит производителя, название, калибр, тип пули, тип патрона, опциональные значения навески пороха и массы пули в гранах (стандартная единица в боеприпасной номенклатуре), а также примечания. Уникальное ограничение на тройку (`manufacturer, name, caliber`) исключает дублирование. Индексы созданы по каждому из основных полей фильтрации, перечисленных в техническом задании.

Таблица `ammo_stocks` хранит текущий остаток патронов для пары (`user_id, ammo_type_id`). Это таблица материализованного агрегата: она содержит сумму всех транзакций для данной пары и поддерживается в актуальном состоянии той же транзакцией базы данных, которая записывает соответствующую запись в журнал транзакций. Денормализация оправдана тем, что типичный запрос "мой текущий запас всех типов боеприпасов" встречается на порядки чаще, чем модифицирующие операции, и его эффективное выполнение через `GROUP BY` по полному журналу транзакций стало бы узким местом по мере роста объёма данных. Ограничение `CHECK (quantity >= 0)` гарантирует, что кэшированное значение никогда не примет отрицательное значение: попытка списать больше патронов, чем имеется в наличии, приводит к откату всей

транзакции на уровне СУБД, обеспечивая физическую невозможность рассогласования кэша и журнала.

Таблица `ammo_transactions` — неизменяемый журнал всех операций с патронами. Каждая запись содержит подписанное значение `delta` (положительное — приобретение, отрицательное — расход), опциональную ссылку на единицу оружия (типична для операций расхода — "израсходовал 50 патронов из своей винтовки"), метку времени, к которой пользователь относит событие (в отличие от технической метки создания записи), и опциональные примечания. Ограничение `CHECK (delta <> 0)` исключает запись пустых операций. Каскадное удаление при удалении пользователя обеспечивает целостность; для типа боеприпасов используется `ON DELETE RESTRICT` — администратор не может удалить справочную запись, на которую ссылается хотя бы одна пользовательская транзакция.

Индексы поддерживают три основных сценария доступа: `(user_id, occurred_at DESC)` — для истории операций пользователя, `(user_id, ammo_type_id)` — для агрегации по типу, и частичный индекс `(user_id, gun_id) WHERE gun_id IS NOT NULL` — для статистики использования конкретного оружия.

2.3.6 Баллистические профили и профили прицелов

Таблица `ballistic_profiles` хранит именованные пресеты баллистических параметров (комбинация "оружие + патрон + прицел"), которые пользователь может многократно использовать для расчётов траекторий. Вложенные структуры (параметры пули, прицела, атмосферные параметры) сериализуются в колонки типа `JSONB`. Этот подход избавляет от необходимости создания широкой таблицы с двумя десятками числовых колонок, изменение состава которой потребовало бы миграцию схемы при каждом добавлении нового параметра. Уникальное ограничение на пару `(user_id, name)` исключает дублирование имён в пределах одного пользователя.

Таблица `scope_profiles` хранит конфигурацию оптических прицелов пользователя: название, опциональную привязку к единице оружия, единицу регулировки (MOA, IPHY или MIL), цену клика, опциональные максимальные

пределы регулировки по вертикали и горизонтали, высоту крепления прицела над осью ствола.

Обе таблицы используют каскадное удаление при удалении пользователя.

2.3.7 Нормативно-правовые акты

Подсистема нормативно-правовых актов реализована тремя таблицами.

Таблица `law_categories` — иерархический справочник категорий нормативных актов (хранение, транспортировка, охота, самооборона и т. п.). Колонка `parent_id` поддерживает древовидную структуру (например, "Охота" может содержать подкатегории "Огнестрельное охотничье оружие" и "Пневматическое охотничье оружие"). Колонка `sort_order` задаёт порядок отображения в интерфейсе.

Таблица `laws` содержит текущие версии законов: стабильный машинно-читаемый ключ (`law_key`, например `fz-150-1996`), заголовок, краткое резюме, полный текст, регион действия, ссылку на категорию, массив тегов (тип `TEXT[]`), номер текущей версии, дату вступления в силу. Закрытый словарь тегов (транспортировка, ношение, хранение, охота, самооборона, спорт, лицензия, наследование, проверка, наказание, прочее) валидируется ограничением `CHECK` на уровне базы данных через оператор `tags <@ ARRAY[...]`.

Ключевой особенностью таблицы является колонка `search_vector` типа `tsvector`, поддерживаемая триггером `laws_search_vector_trigger`. При вставке или обновлении заголовка, резюме или текста закона триггер автоматически регенерирует поисковый вектор, объединяя результаты `to_tsvector('russian', ...)` и `to_tsvector('english', ...)` с разными весами: заголовок получает вес А, резюме — В, основной текст — С. Это позволяет ранжировать результаты поиска по релевантности: совпадения в заголовке оцениваются выше совпадений в тексте. Объединение русской и английской конфигураций позволяет выполнять поиск по запросам на любом из двух языков (что важно для нормативных актов, цитирующих международные договоры и иностранные термины).

Для поискового вектора создан индекс типа `GIN`, обеспечивающий эффективное выполнение запросов с оператором `@@`. Аналогичный `GIN`-индекс

создан для массива тегов, что позволяет эффективно вычислять пересечения (`tags && ARRAY[...]`) и включения (`tags @> ARRAY[...]`).

Таблица `law_versions` хранит исторические снимки законов. Заполняется триггером `laws_snapshot_trigger`, который при изменении содержательных полей (заголовков, текст, резюме, теги, категория, дата вступления в силу) автоматически вставляет копию старой строки таблицы `laws` в `law_versions` под старым номером версии, после чего инкрементирует `current_version` в основной таблице. Изменения, не затрагивающие содержание (например, только метку времени последнего обновления), не приводят к созданию новой версии. Такая реализация версионирования на уровне базы данных гарантирует, что ни один путь записи через приложение не сможет пропустить снимок: версионирование является инвариантом схемы, а не контракта прикладного кода.

Все таблицы используют тип `UUID` в качестве первичного ключа (за исключением `audit_log`, использующей `BIGSERIAL` для эффективного добавления записей). Каскадное удаление при удалении пользователя применяется ко всем пользовательским данным, что гарантирует выполнение требования о возможности полного удаления учётной записи пользователя без оставления "осиротевших" записей в системе.

Все индексы спроектированы под конкретные сценарии запросов, выявленные на этапе проектирования вариантов использования: индексы по `user_id` для запросов в рамках одного пользователя, составные индексы для совместной фильтрации по нескольким атрибутам, GIN-индексы для поиска по массивам и полнотекстового поиска, частичные индексы для запросов, обращающихся только к подмножеству строк.

Схема базы данных серверной части приложения «Dryfire»

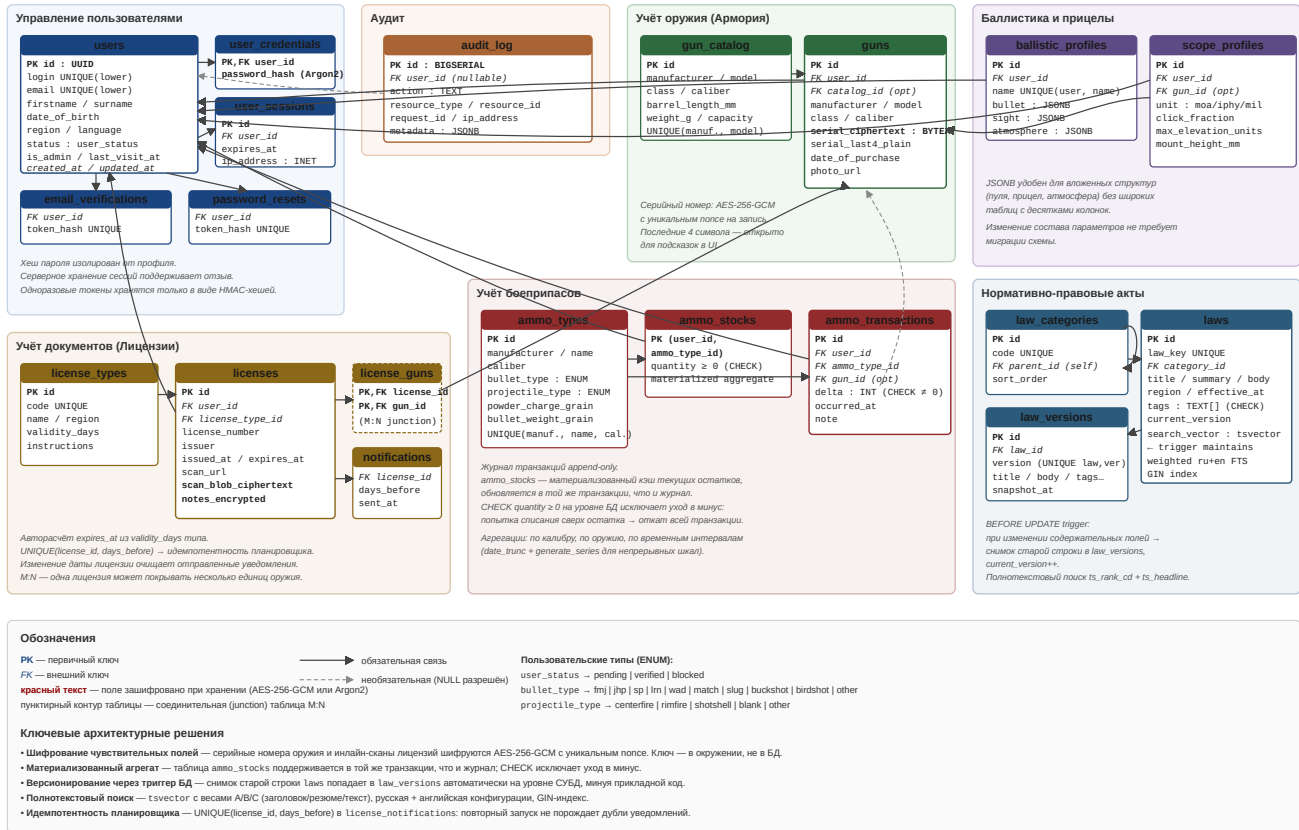


Рисунок 2. Диаграмма базы данных

Заключение

В рамках настоящей курсовой работы разработана серверная часть кроссплатформенного приложения «Dryfire», предназначенного для гражданских владельцев огнестрельного оружия. Все задачи, поставленные во введении, были выполнены в полном объёме.

В **первой главе** проведён анализ актуальности разработки. Установлено, что в Российской Федерации насчитывается порядка 4,3 миллиона активных владельцев гражданского оружия и около 6,5 миллионов единиц оружия. Существующие решения для учёта оружия, документов и боеприпасов либо ориентированы на профессиональных пользователей и обладают избыточной сложностью, либо представляют собой узкоспециализированные инструменты (например, баллистические калькуляторы), не покрывающие всего спектра потребностей владельца. Отсутствие единого комплексного решения, объединяющего ведение электронного арсенала, контроль сроков действия документов, учёт боеприпасов, баллистические расчёты и доступ к актуальной нормативно-правовой базе, подтверждает обоснованность разработки.

Также в первой главе проведён анализ современных технологий разработки серверных приложений. Обоснован выбор языка программирования Rust, обеспечивающего безопасность работы с памятью без сборщика мусора и высокую производительность сравнимую с C/C++. Обоснован выбор веб-фреймворка Axum (асинхронного, типобезопасного), библиотеки SQLx для работы с PostgreSQL (с проверкой SQL-запросов во время компиляции), асинхронной среды выполнения Tokio, а также криптографических библиотек Argon2 для хеширования паролей и AES-GCM для шифрования чувствительных полей.

Проведён анализ нормативно-правовой базы в сфере оборота гражданского оружия Российской Федерации, включая Федеральный закон «Об оружии» (ФЗ-150), приказы МВД и Росгвардии, регулирующие хранение, ношение, транспортировку и использование оружия. Проанализированы доступные API для программного получения нормативно-правовых актов из открытых источников (pravo.gov.ru, КонсультантПлюс), что определило архитектуру подсистемы фоновой загрузки и индексирования нормативных актов.

Во **второй главе** на основе результатов анализа определён функционал разрабатываемого приложения и реализована серверная часть приложения. Сформулированы функциональные требования по семи предметным областям: управление пользователями, ведение арсенала оружия, учёт документов и разрешений, учёт боеприпасов, баллистические расчёты, расчёт поправок оптических прицелов, доступ к нормативно-правовым актам с возможностью полнотекстового поиска.

Архитектура серверной части реализована в соответствии с принципами гексагональной (портово-адаптерной) архитектуры, обеспечивающей строгое разделение между бизнес-логикой и инфраструктурными деталями. Кодовая база разделена на четыре слоя — доменный, прикладной, инфраструктурный и слой контроллеров — с однонаправленными зависимостями, направленными от внешних слоёв к внутреннему ядру. Каждая из семи предметных областей реализована по единому шаблону «сущность → порт репозитория → вариант использования → контроллер», что обеспечивает однородность кодовой базы и предсказуемость её роста.

Спроектирована и реализована схема базы данных, включающая 18 таблиц, 3 пользовательских перечисляемых типа, а также набор индексов и триггеров, обслуживающих специфические сценарии доступа к данным. Применён ряд нетривиальных приёмов: материализованный агрегат остатков боеприпасов, поддерживаемый в одной транзакции с журналом операций; автоматическое версионирование нормативных актов посредством триггеров уровня СУБД; полнотекстовый поиск с поддержкой русского и английского языков на базе типа `tsvector` и GIN-индексов; шифрование чувствительных полей (серийных номеров оружия и сканов документов) алгоритмом AES-256-GCM с уникальным поппсе на каждую операцию записи.

Реализовано полноценное HTTP API, охватывающее все функциональные требования технического задания. Аутентификация пользователей построена на сессионных токенах с серверным хранилищем, что позволяет, в отличие от полностью stateless-подхода, принудительно отзываться сессии при компрометации. Применены актуальные практики обеспечения безопасности: хеширование паролей алгоритмом Argon2id, защита от атак по времени в обработчике входа, параметризованные SQL-запросы, защита от SQL-инъекций, изоляция данных по

идентификатору пользователя на уровне репозитория, журнал аудита с фиксацией IP-адреса и идентификатора запроса, стандартизированный формат ответов об ошибках.

Реализована подсистема фоновых задач, отправляющая пользователям напоминания об окончании сроков действия документов за 90, 60, 45, 30 и 14 дней до истечения. Идемпотентность планировщика обеспечена на уровне базы данных через уникальное ограничение на пару значений (идентификатор документа, количество дней до окончания), что гарантирует невозможность повторной отправки уже отправленных уведомлений даже при многократных перезапусках планировщика или дублирующих циклах обработки.

Практическая значимость работы заключается в создании готовой к развёртыванию серверной части приложения, обладающей всеми архитектурными свойствами, необходимыми для эксплуатации в продуктивной среде: модульностью, расширяемостью, тестируемостью, безопасностью обработки чувствительных данных. Применённый архитектурный подход допускает дальнейшее развитие системы — добавление новых предметных областей, замену конкретных технологий инфраструктуры, подключение мобильных и веб-клиентов — без необходимости пересмотра ядра бизнес-логики.

Возможные направления дальнейшего развития включают: подключение фоновой подсистемы загрузки нормативно-правовых актов из открытых источников; интеграцию с провайдерами push-уведомлений (FCM, APNs) в дополнение к электронной почте; реализацию загрузки файлов (сканов документов и фотографий оружия) в объектное хранилище; автоматическое формирование документации API в формате OpenAPI 3.0; реализацию ограничения частоты запросов (rate limiting) для эндпоинтов аутентификации; настройку конвейера непрерывной интеграции и доставки (CI/CD).

Таким образом, цель курсовой работы — разработка серверной части кроссплатформенного приложения «Dryfire» — достигнута. Полученное решение соответствует всем функциональным и нефункциональным требованиям, обладает архитектурными свойствами, необходимыми для развития в продуктивный продукт, и образует прочную основу для последующей разработки клиентских приложений.

Краткий глоссарий основных терминов

Архитектура и проектирование

Гексагональная архитектура (портово-адаптерная архитектура, Clean Architecture) — архитектурный стиль, при котором бизнес-логика приложения изолирована от деталей реализации инфраструктуры с помощью интерфейсов («портов»), реализуемых внешними компонентами («адаптерами»).

Порт (port) — интерфейс, определённый доменным слоем и описывающий контракт, который должен выполнить инфраструктурный компонент. В Rust реализуется через `trait`.

Адаптер (adapter) — конкретная реализация порта для определённой технологии (например, реализация репозитория для PostgreSQL).

Сущность (entity) — объект предметной области с уникальным идентификатором и набором инвариантов (например, `User`, `Gun`, `License`).

Вариант использования (use case) — компонент прикладного слоя, оркеструющий выполнение одного бизнес-сценария: загрузку сущностей, вызов доменной логики и фиксацию изменений.

Репозиторий (repository) — объект, инкапсулирующий доступ к хранилищу данных и предоставляющий типизированный API для работы с сущностями.

DTO (Data Transfer Object) — объект для передачи данных между слоями приложения, не содержащий бизнес-логики.

Технологии разработки

Rust — компилируемый системный язык программирования с гарантиями безопасности работы с памятью без сборщика мусора, обеспечиваемыми механизмом владения (ownership) и заимствования (borrowing) на этапе компиляции.

Cargo — пакетный менеджер и система сборки языка Rust.

Crate (крейт) — единица распространения исходного кода в экосистеме Rust; библиотека или исполняемое приложение.

Tokio — асинхронная среда выполнения для языка Rust, обеспечивающая многопоточность и неблокирующий ввод-вывод.

Axum — асинхронный веб-фреймворк для языка Rust, построенный на базе библиотеки tower и среды выполнения Tokio.

SQLx — асинхронная библиотека Rust для работы с реляционными СУБД, поддерживающая проверку SQL-запросов на этапе компиляции.

Serde — библиотека сериализации и десериализации данных в Rust для различных форматов (JSON, TOML и др.).

Trait — механизм абстракции в языке Rust, аналогичный интерфейсам в других языках; используется для определения портов в гексагональной архитектуре.

Async / await — синтаксические конструкции языка Rust для написания асинхронного кода в стиле, близком к синхронному.

Базы данных

СУБД — система управления базами данных.

PostgreSQL — объектно-реляционная СУБД с открытым исходным кодом, поддерживающая расширенные типы данных (JSONB, массивы, перечисления, полнотекстовый поиск).

ENUM — пользовательский перечисляемый тип в PostgreSQL, ограничивающий допустимые значения колонки фиксированным набором.

UUID (Universally Unique Identifier) — 128-битный универсально уникальный идентификатор, используемый в качестве первичного ключа сущностей.

JSONB — двоичный формат хранения данных JSON в PostgreSQL, поддерживающий индексирование и операции запроса.

INET — тип данных PostgreSQL для хранения IP-адресов (IPv4 и IPv6).

Триггер — функция, автоматически выполняемая СУБД при наступлении определённого события (например, перед обновлением строки таблицы).

Миграция (database migration) — версионированное изменение схемы базы данных, применяемое автоматически при развёртывании приложения.

Внешний ключ (foreign key, FK) — ограничение целостности, требующее, чтобы значение в одной таблице соответствовало значению первичного ключа в другой.

Первичный ключ (primary key, PK) — колонка или комбинация колонок, однозначно идентифицирующая строку в таблице.

Каскадное удаление (ON DELETE CASCADE) — поведение, при котором удаление строки в родительской таблице автоматически удаляет связанные строки в дочерних таблицах.

Материализованный агрегат — таблица, в которой хранится предвычисленный результат агрегирующей операции (например, текущий остаток патронов), поддерживаемая в актуальном состоянии той же транзакцией, что и источник данных.

tsvector / tsquery — типы данных PostgreSQL, обеспечивающие полнотекстовый поиск: **tsvector** хранит нормализованное представление текста, **tsquery** — поисковый запрос.

GIN-индекс (Generalized Inverted Index) — тип индекса PostgreSQL, эффективный для типов данных с множественными значениями (массивы, **tsvector**).

Безопасность

Argon2id — алгоритм хеширования паролей, победитель Password Hashing Competition 2015, рекомендованный OWASP. Замедляет атаки полным перебором за счёт высоких требований к памяти и времени вычисления.

HMAC (Hash-based Message Authentication Code) — алгоритм построения кодов аутентификации сообщения на основе криптографических хеш-функций. В работе используется HMAC-SHA256.

SHA-256 — криптографическая хеш-функция семейства SHA-2, формирующая 256-битный выход.

AES-GCM (Advanced Encryption Standard в режиме Galois/Counter Mode) — симметричный алгоритм аутентифицированного шифрования; в работе используется с ключом 256 бит.

Nonce (number used once) — уникальное случайное значение, используемое однократно в криптографическом протоколе. В AES-GCM обеспечивает безопасность многократного шифрования одного и того же открытого текста на одном ключе.

SQL-инъекция — атака, при которой злоумышленник внедряет SQL-конструкции в пользовательский ввод, исполняемый базой данных. Защищается параметризованными запросами.

Атака по времени (timing attack) — атака, основанная на анализе времени выполнения операций для извлечения секретной информации.

Атака полным перебором (brute-force attack) — попытка подобрать пароль или ключ путём проверки всех возможных значений.

TLS (Transport Layer Security) — криптографический протокол, обеспечивающий конфиденциальность и целостность данных при сетевом взаимодействии.

Соль (salt) — случайное значение, добавляемое к паролю перед хешированием для защиты от атак по радужным таблицам.

Журнал аудита (audit log) — таблица, фиксирующая все значимые действия пользователей и системы для целей расследования инцидентов и контроля доступа.

Сеть и API

API (Application Programming Interface) — программный интерфейс взаимодействия между компонентами или системами.

REST (Representational State Transfer) — архитектурный стиль построения API на основе протокола HTTP, использующий стандартные методы (GET, POST, PUT, DELETE) для операций с ресурсами.

HTTPS — протокол HTTP, защищённый шифрованием TLS.

JSON (JavaScript Object Notation) — текстовый формат обмена данными, использующийся в качестве основного формата представления данных в API.

OpenAPI (ранее Swagger) — спецификация описания REST API в машиночитаемом формате, поддерживающая автоматическую генерацию документации.

Middleware — слой промежуточной обработки запроса между точкой приёма HTTP-запроса и его обработчиком (например, проверка авторизации, логирование, ограничение частоты запросов).

CORS (Cross-Origin Resource Sharing) — механизм, позволяющий веб-странице запрашивать ресурсы с домена, отличного от домена самой страницы.

SMTP (Simple Mail Transfer Protocol) — протокол передачи электронной почты.

FCM / APNs (Firebase Cloud Messaging / Apple Push Notification service) — сервисы доставки push-уведомлений на мобильные устройства Android и iOS соответственно.

Bearer-токен — формат токена авторизации, передаваемого в HTTP-заголовке `Authorization: Bearer <token>`.

Предметная область (огнестрельное оружие)

Гражданское оружие — оружие, разрешённое к обороту среди гражданских лиц законодательством Российской Федерации в соответствии с Федеральным законом «Об оружии» (ФЗ-150).

Калибр (caliber) — диаметр канала ствола огнестрельного оружия и соответствующего ему боеприпаса, выраженный в миллиметрах или долях дюйма (например, 9 мм, .308 Win, .22 LR).

Лицензия на оружие — документ, разрешающий гражданину приобретение, хранение и использование определённого вида гражданского оружия.

Разрешение на хранение и ношение (РОХа) — документ, разрешающий хранение и ношение конкретной единицы оружия владельцем.

Боеприпас (патрон) — устройство, объединяющее метательный заряд, пулю и средство воспламенения, предназначенное для использования в огнестрельном оружии.

Гильза — оболочка патрона, удерживающая порох и пулю; имеет дно с капсюлем.

Капсюль — элемент патрона, инициирующий воспламенение порохового заряда при ударе бойка.

Пуля — метаемый элемент патрона огнестрельного оружия (для гладкоствольного оружия — пуля, картечь или дробь).

FMJ (Full Metal Jacket) — оболочечная пуля со свинцовым сердечником, покрытая металлической оболочкой.

JHP (Jacketed Hollow Point) — оболочечная экспансивная пуля с углублением в передней части, обеспечивающим расширение при попадании в цель.

Гран (grain) — единица массы, используемая в боеприпасной номенклатуре; 1 гран $\approx$ 64,8 мг.

Навеска пороха — масса метательного заряда патрона.

Огнестрельное оружие центрального воспламенения (centerfire) — оружие, в патронах которого капсюль расположен в центре дна гильзы.

Оружие кольцевого воспламенения (rimfire) — оружие, в патронах которого инициирующий состав расположен по краю донца гильзы.

Баллистика и оптика

Баллистика — раздел механики, изучающий движение метаемых тел; в применении к огнестрельному оружию — движение пули.

Внешняя баллистика — движение пули от момента покидания ствола до встречи с целью; определяется начальной скоростью, баллистическим коэффициентом, гравитацией, сопротивлением воздуха и ветром.

Баллистический коэффициент (BC) — числовая характеристика пули, описывающая её способность преодолевать сопротивление воздуха; чем выше, тем лучше пуля сохраняет скорость на дальней дистанции.

Модель G1 — стандартная аэродинамическая модель для расчёта внешней баллистики, основанная на эталонной пуле определённой формы; наиболее распространённая в боеприпасной номенклатуре.

Метод Рунге — Кутты 4-го порядка (RK4) — численный метод решения обыкновенных дифференциальных уравнений, используемый для интегрирования уравнений движения пули.

Прицел (sight, scope) — устройство для прицеливания при стрельбе; в работе рассматривается оптический прицел с регулируемой сеткой.

Поправка прицела — отклонение точки прицеливания от точки попадания, выражаемое в угловых единицах.

Клик (click) — единичный шаг регулировки сетки прицела.

МОА (Minute of Angle, угловая минута) — единица измерения углов, равная $1/60$ градуса; на дистанции 100 ярдов соответствует приблизительно 1 дюйму, на дистанции 100 м — приблизительно 2,9 см.

IPHY (Inch Per Hundred Yards) — единица измерения, при которой 1 единица соответствует ровно 1 дюйму на дистанции 100 ярдов; иногда смешивается с МОА, однако отличается на $\sim 4,7\%$.

MIL (миллирадиан) — единица измерения углов, равная $1/1000$ радиана; на дистанции 100 м соответствует 10 см.

Пристрелка (zeroing) — настройка прицела таким образом, чтобы средняя точка попадания совпадала с точкой прицеливания на заданной дистанции.

Высота крепления прицела (mount height) — расстояние от оси канала ствола до оптической оси прицела, влияющее на расчёт баллистической поправки.

Список использованных литературных источников

1. Klabnik, S. The Rust Programming Language : 2nd Edition / Steve Klabnik, Carol Nichols. — San Francisco : No Starch Press, 2023. — 560 p. — ISBN 978-1-7185-0310-6. — Текст : электронный. — URL: <https://doc.rust-lang.org/book/> (дата обращения: 15.05.2026).
2. Evans, E. Domain-Driven Design: Tackling Complexity in the Heart of Software / Eric Evans. — Boston : Addison-Wesley Professional, 2003. — 560 p. — ISBN 0-321-12521-5. — Текст : непосредственный.
3. Biryukov, A. Argon2: the memory-hard function for password hashing and other applications / Alex Biryukov, Daniel Dinu, Dmitry Khovratovich. — University of Luxembourg, 2015. — Version 1.2.1. — Текст : электронный. — URL: <https://www.password-hashing.net/argon2-specs.pdf> (дата обращения: 12.05.2026).
4. RFC 9106. Argon2 Memory-Hard Function for Password Hashing and Proof-of-Work Applications / A. Biryukov, D. Dinu, D. Khovratovich, S. Josefsson. — Internet Research Task Force (IRTF), September 2021. — Текст : электронный. — URL: <https://www.rfc-editor.org/rfc/rfc9106.html> (дата обращения: 12.05.2026).
5. McCoy, R. L. Modern Exterior Ballistics: The Launch and Flight Dynamics of Symmetric Projectiles : 2nd Edition / Robert L. McCoy. — Atglen : Schiffer Publishing, 2012. — 328 p. — ISBN 978-0-7643-3825-0. — Текст : непосредственный.
6. Федеральный закон от 13.12.1996 № 150-ФЗ «Об оружии» (ред. от 15.12.2025) // КонсультантПлюс : справочно-правовая система. — URL: https://www.consultant.ru/document/cons_doc_LAW_12679/ (дата обращения: 20.05.2026). — Текст : электронный.
7. Документация PostgreSQL 16 / The PostgreSQL Global Development Group. — Текст : электронный. — URL: <https://www.postgresql.org/docs/16/index.html> (дата обращения: 18.05.2026).
8. Axum: ergonomic and modular web framework built with Tokio, Tower, and Hyper : документация / Tokio Project. — Текст : электронный. — URL: <https://docs.rs/axum/latest/axum/> (дата обращения: 20.05.2026).

9. SQLx: The Rust SQL Toolkit : документация / launchbadge. — Текст : электронный. — URL: <https://docs.rs/sqlx/latest/sqlx/> (дата обращения: 20.05.2026).

10. The Tokio Project: An asynchronous runtime for the Rust programming language : документация и руководство. — Текст : электронный. — URL: <https://tokio.rs/tokio/tutorial> (дата обращения: 19.05.2026).